Lab Agentic Driving Coach — De A à Z

Guide technique complet et support de présentation orale pour l'enseignant

Cours : CSE 494 / CSE 598 (ISCPS) **Élève :** Binôme / Groupe 01 **Sujet :** Systèmes Cyber-Physiques & LLM Temps Réel **Framework :** Xronos Python SDK 0.13.1

1. Le Projet en 30 secondes : ce qu'on cherche à prouver

Ce lab étudie l'intégration d'un **modèle d'intelligence artificielle (LLM)** dans la boucle de contrôle critique d'un véhicule autonome approchant d'un panneau *STOP*.

Le problème fondamental : Un modèle de langage comme *Llama-3.2* met du temps à réfléchir (latence variable de 300 ms à plus de 1,5 s). Or, une voiture qui roule à 10 m/s (36 km/h) parcourt 1 mètre tous les 100 ms. Si le système attend passivement la réponse du LLM pour décider de freiner, la voiture s'écrase contre l'obstacle.

La solution démontrée : L'utilisation d'une architecture orientée **réacteurs Xronos** avec une **course d'événements (race)** et une **politique de secours déterministe (fallback)**. Si le LLM ne répond pas avant une échéance stricte (deadline), le système l'ignore instantanément et applique les règles de sécurité déterministes.

2. Les 7 Notions Essentielles (à savoir expliquer)

Concept	Explication simple en français	Rôle dans le projet
Réacteur & Réaction	Composant logiciel autonome qui reçoit des messages sur des ports, possède son état interne et exécute du code (réaction) sur événement.	5 réacteurs : `Driver`, `Car`, `RoadEnvironment`, `Coach`, `Recorder`.
Temps Logique	Horloge mathématique exacte et déterministe. Deux événements prévus à $t = 100\text{ ms}$ ont exactement le même tag, sans retard ni avance.	Permet des simulations 100% reproductibles, indépendamment de la puissance du processeur.
Temps Réel (Wall-clock)	L'horloge physique du monde réel (secondes de notre montre et du processeur).	Sert à mesurer la véritable durée d'exécution du LLM ou d'un calcul.
Latence (Latency)	Temps réel que met le LLM pour générer sa réponse (ex: 350 ms).	Si elle dépasse la deadline, la réponse est dite périmée (late).
Échéance (Deadline)	Date limite temporelle stricte au-delà de laquelle une décision devient inutile ou dangereuse.	Fixée à 300 ms en local / poste de travail et 1700 ms sur supercalculateur Sol.
Lag & Slack	**Lag :** Retard du système réel sur le temps logique. **Slack :** Marge restante avant l'échéance ($T_{\text{deadline}} - T_{\text{réel}}$).	Si slack < 0, on a raté la deadline. Si lag est grand, le monde physique a pris du retard.
Secours (Fallback)	Mécanisme réflexe codé en dur (règles physiques) qui s'exécute automatiquement en cas de panne ou retard du LLM.	Garantit que le véhicule freine à temps même si le LLM plante ou est trop lent.

3. Ce que nous avons fait concrètement, de A à Z

Partie 0 — Vérification de l'environnement

- **Objectif :** S'assurer que le runtime Python, Xronos et les outils sont conformes.
- **Ce que nous avons exécuté :** `python scripts/check_environment.py` et `coach doctor`.
- **Résultat :** Génération du fichier officiel `results/environment.txt`. L'environnement Linux (WSL2 Ubuntu 24.04, Python 3.12.3, Xronos 0.13.1, Matplotlib 3.11.2) a validé tous les tests de conformité.

Partie 1 — Timing Xronos et gestion des délais (Exemples 01 à 05)

- **Exemple 01 & 02 :** Validation du démarrage des réacteurs et des ports périodiques (100 ms). Nous avons prouvé que les timestamps affichés sont logiques car ils progressent sans aucune milliseconde de gigue.
- **Exemple 03 (Délai logique) :**
  - Le délai de 500 ms est configuré à la ligne 65 (`env.connect(..., delay=timedelta(milliseconds=500))`).
  - **Test de prédiction :** Nous avons modifié ce délai à 300 ms. Nous avions prédit que les réceptions décaleraient exactement à $t = 300, 550, 800, 1050, 1300\text{ ms}$. L'exécution a validé notre prédiction à 100%, prouvant que Xronos calcule les délais par arithmétique de tags dans sa file d'attente.
- **Exemple 04 (Lag & Slack) :** Quand un composant prend 80 ms pour une échéance de 50 ms, son slack devient négatif (-31.5 ms) et le lag grimpe à 31.6 ms. Dès le tick suivant, le système récupère instantanément un lag quasi-nul (0.38 ms).

- **Exemple 05 (Fallback rétroactif vs Course d'événements) :**
  - **Phase A (Anti-pattern) :** L'appel modèle est synchrone et bloquant (1.2 s). Le moniteur subit un lag énorme de 1 101 ms et le fallback se déclenche avec **800 ms de retard** sur la deadline. Il ne protège rien.
  - **Phase B (Pattern du projet) :** Le handler lance la requête et rend immédiatement la main. Le runtime fait la course entre deux timers programmables (`on_response` et `on_deadline`). Le fallback se déclenche **exactement à temps** à $t = 600\text{ ms}$ (lag de 0.5 ms), et la réponse tardive arrivée à 1400 ms est proprement jetée.

Partie 2 — Simulations déterministes de référence

- **Simulations lancées :** `results/rule-a`, `results/rule-b` et `results/replay`.
- **Preuve de déterminisme :** La comparaison champ par champ de `results/rule-a/summary.json` et `results/rule-b/summary.json` montre une égalité stricte sur toutes les métriques physiques et logiques (vitesse, arrêts, actionnements). Seuls `run_id` (généré aléatoirement) et `xronos_lag` (mesure matérielle) changent.
- **Résultat d'arrêt :** Le véhicule s'arrête parfaitement 4.522 m avant le panneau (`stop_position_error_m = -4.522 m`, `v = 0.0 m/s`).
- **Séquence d'armement du freinage d'urgence :**
  1. $t = 6700\text{ ms}$: Le coach émet son 1er jeton `ACTUATE`. Le Planner s'arme et émet un avertissement vocal ("Brake now").
  2. $t = 6800\text{ ms}$: Au tick suivant (100 ms plus tard), recevant à nouveau `ACTUATE`, le Planner armé déclenche l'actionneur physique `EMERGENCY_BRAKING`.
  3. $t = 7000\text{ ms}$: Après les 200 ms de délai de transmission vers la voiture, le freinage d'urgence est appliqué à la physique du véhicule.
- **Modification du réacteur Planner :** Nous avons ajouté une instruction verbale lors de la désescalade (`[VERBAL] NONE | Back within the safe band.`). Prédiction confirmée : exactement 1 ligne ajoutée à $t = 4600\text{ ms}$, sans aucun changement sur le comportement physique du véhicule (le conducteur rejouant une trace prédéfinie).

Partie 3 — Évaluation des modèles LLM (Llama 3.2 1B vs 3B)

Nous avons analysé le comportement sous une échéance stricte de 300 ms à l'aide des traces certifiées du cours (exécutées via `results/replay-1b-300` et `results/replay-3b-300`).

Modèle	Latence Médiane	Taux Raté Deadline	Réponses Malformées	Faux Négatifs Dangereux	Résultat d'arrêt
Llama-3.2 1B	349.8 ms	85.7 %	0	**1 (DANGEREUX)**	Arrêt dépassé (+20.1 m)
Llama-3.2 3B	362.8 ms	95.2 %	1	**0 (SÛR)**	Arrêt dépassé (+20.0 m)

> **Le compromis Latence vs Qualité mis en évidence :**
> Le petit modèle (1B) répond un peu plus vite, mais commet une faute grave : il oublie de demander le freinage alors que l'oracle de sécurité l'imposait (1 faux négatif non sécurisé). Le modèle 3B comprend mieux les règles de sécurité (0 faux négatif), mais sa latence dépasse l'échéance de 300 ms dans 95% des cas, forçant le système à utiliser le fallback.

Partie 4 — Comparaison des profils conducteur (Débutant vs Avancé)

- **Commande exécutée :** `coach compare-behaviors --drivers beginner advanced --repetitions 3`.
- **Résultats clés :**
  - Le conducteur **débutant** ne freine pas de lui-même : il passe **3.1 secondes en infraction** de vitesse et nécessite **11 freinages d'urgence** du coach pour s'arrêter à 4.52 m du stop.
  - Le conducteur **avancé** ralentit de manière autonome : il ne passe que **2.4 secondes en infraction** et ne nécessite que **8 freinages d'urgence**, s'arrêtant avec une marge confortable de **13.08 mètres** avant le stop.
- **Artéfact produit :** Graphique comparatif officiel `results/behavior-comparison/comparison.png`.

Partie 5 — Extension Graduate CSE 598 (Option B : Traces générées)

- **Notre contribution :** Développement du script `scripts/generate_driver_trace.py` qui produit des profils conducteurs stochastiques et réalistes en contrôlant la graine du générateur (`random.Random(seed)`).
- **Expérience :** Génération de 3 profils avec les graines 42, 101 et 2024, puis évaluation contre le coach de règles déterministe.
- **Conclusion de l'extension :** Même face à des conducteurs aux comportements très irréguliers (début de freinage retardé, hésitations), la politique réflexe du coach a garanti un arrêt sans collision dans 100% des cas (arrêt entre 1.33 m et 2.53 m du stop), au prix d'un nombre accru d'interventions d'urgence (12 à 13).

4. Fiche « Questions Pièges » du Professeur

Voici les questions typiques qu'un enseignant pose lors de la soutenance, et exactement ce que vous devez lui répondre :

> **Q1 : « Pourquoi avez-vous besoin de Xronos ? Pourquoi ne pas simplement faire un script Python avec des threads et des sleep() ? »**
>
> **Réponse :** Parce que les threads et `time.sleep()` en Python dépendent de l'OS et du GIL, ce qui introduit de la gigue (jitter) non déterministe. Deux exécutions donneraient des trajectoires physiques différentes. Xronos utilise la sémantique des réacteurs avec un **temps logique déterministe** : l'exécution est reproductible au microseconde près, tout en permettant d'exécuter des simulations accélérées (`--fast`) sans perdre la validité des modèles physiques.

Q2 : « Dans l'exemple 05, quelle est la différence structurelle clé entre la Phase A et la Phase B ? »

Réponse : Dans la Phase A, le handler attend la réponse du modèle de manière synchrone et bloquante. C'est un *anti-pattern* : le runtime est paralysé et ne peut pas faire respecter l'échéance. Dans la Phase B, la requête est asynchrone et le runtime gère une **course d'événements** entre deux timers : la réponse et l'échéance. Si l'échéance arrive en premier, le fallback tire instantanément.

Q3 : « Pourquoi observe-t-on un délai de 300 ms entre la première décision d'actionner et le moment où la voiture commence à freiner ? »

Réponse : Ce délai se décompose en deux mécanismes distincts :

1. **100 ms d'armement du Planner :** Le premier jeton `ACTUATE` (à 6700 ms) passe le Planner en mode armé et émet une alerte verbale, mais ne freine pas encore. Il faut un 2e jeton consécutif (à 6800 ms) pour déclencher l'action physique.
2. **200 ms de délai de transmission logique :** La connexion entre le coach et la voiture (`coach.actuate -> car.actuate_in`) a un délai logique déclaré de 200 ms. L'action émise à 6800 ms s'applique donc à la physique de la voiture exactement à 7000 ms.

Q4 : « Quel modèle préférez-vous entre Llama-3.2:1B et 3B pour ce système critique ? »

Réponse : Aucun des deux ne peut être utilisé seul sans garde-fous. Le modèle 1B est dangereux car il génère des faux négatifs (il omet d'avertir du danger). Le modèle 3B est plus fiable sur le plan sémantique mais trop lent sur machine standard, ce qui entraîne des dépassements de deadline. Cela démontre scientifiquement qu'un système cyber-physique autonome ne peut pas dépendre uniquement d'un LLM et doit impérativement être doublé d'un fallback réflexe déterministe.

Q5 : « Qu'avez-vous rendu dans votre archive finale ? »

Réponse : L'archive officielle `submission/group01_agentic-driving-coach.zip` (144 fichiers) générée par le script du projet `make_submission.py` . Elle contient notre fichier de réponses complet `answers.md` , l'intégralité du code source, nos scripts d'extension (Option B), et les dossiers complets de résultats (CSV, JSON de métriques et graphiques PNG). Elle exclut rigoureusement tout cache et environnement virtuel.

5. Inventaire des Fichiers Prêts sur votre Machine

- `submission/answers.md` : Toutes vos réponses officielles rédigées en anglais technique.
- `results/environment.txt` : Preuve de validation de l'environnement d'exécution.
- `RAPPORT_EXECUTION.md` : Journal complet des commandes et des tests effectués.
- `submission/group01_agentic-driving-coach.zip` : **L'archive finale à déposer sur Canvas.**
- `GUIDE_ETUDIANT_PRESENTATION.pdf` : Ce document prêt à imprimer pour réviser avant votre présentation.